

Chapter 6: Illuminating Problems

Quality-Diversity and MAP-Elites for Delivery Planning

Nature-Inspired Optimisation for Delivery Problems (2022)

Lecture notes based on the book

2022

Chapter Overview I

This chapter introduces a fundamentally different philosophy for optimisation: rather than searching for *one* best solution, the goal is to produce a *rich map* of many high-quality solutions that span the entire space of possible outcomes. This idea is called **illumination**, and the central algorithm is called **MAP-Elites**.

Why illumination matters for delivery planning

In logistics, a single “optimal” solution may become unusable overnight if a vehicle breaks down, staff call in sick, or a last-minute change of plan is required. Rather than re-running a lengthy optimisation from scratch, a manager who has an *archive* of diverse, high-quality plans can simply pick the plan that best fits today’s constraints—in seconds, not hours. The on-demand economy, where customers expect near-instant delivery, makes this capability increasingly critical.

Chapter road-map:

- **6.1 Introduction** — quality-diversity algorithms and the case for illumination.

- **6.2 Illumination Algorithms — MAP-Elites** — the behaviour space, niches (bins), and the archive data structure.
- **6.3 Using MAP-Elites to Plan Deliveries** — applying MAP-Elites to a multi-mode supermarket delivery problem with nine solution characteristics.
- **6.4 Results** — archive coverage, solution diversity, and interactive filtering via constraints and aspirations.
- **6.5 Conclusions** — when illumination pays off, and design choices that matter.

6.1 Introduction: The Case for Diversity I

Classical optimisation returns a single answer. Multi-objective optimisation returns a *Pareto front*—a set of solutions where no one objective can be improved without worsening another. **Quality-Diversity (QD)** algorithms go further: they seek a *structured collection* of high-quality solutions that covers as much of the behaviour space as possible.

Definition: Quality-Diversity Algorithm

A Quality-Diversity algorithm is an evolutionary algorithm that simultaneously optimises for (1) *quality* (each solution in the collection must be as fit as possible within its region) and (2) *diversity* (the collection must represent as many distinct regions of the solution space as possible).

6.1 Introduction: The Case for Diversity II

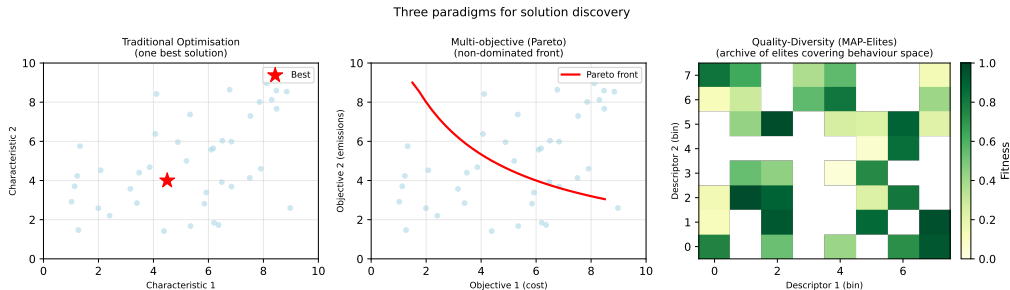


Figure: Three paradigms for solution discovery. Traditional optimisation returns one best point. Multi-objective optimisation returns a Pareto front. Quality-Diversity fills an archive grid with the best solution found for every region of the space.

6.1 Introduction: The Case for Diversity III

The delivery planning context. Consider a supermarket that must plan daily routes for its vans and cargo bikes. The planner cares about many things simultaneously: total cost, staff cost, emissions, the fraction of deliveries made by bike, how many vehicles are needed, and so on. A traditional single-objective algorithm might minimise total cost, but the resulting plan might be environmentally unacceptable. A bi-objective algorithm could trade cost against emissions, but it ignores the other seven concerns. MAP-Elites, the illumination algorithm studied in this chapter, produces an archive of hundreds or thousands of plans, each one the best solution ever found for a particular combination of cost-level and emission-level (and other characteristics). The planner can then *browse* this archive and select the plan that matches today's operational priorities—without waiting for a new optimisation run.

Key Insight: Illumination shifts decision-making

Traditional algorithms make decisions *inside* the optimiser (“which objective do I weight most?”). Illumination algorithms shift this decision *outside* to the human expert, who can apply domain knowledge *after* the archive has been built. This is especially valuable when priorities change frequently—as they do in on-demand logistics.

6.2 MAP-Elites: Core Idea I

MAP-Elites stands for **Map Archive of Phenotypic Elites**. It was introduced by Mouret and Clune (2015) in the paper *Illuminating Search Spaces by Mapping Elites*. The central idea is to maintain a structured map (or grid) of solutions, where each *cell* in the grid holds the single best solution ever discovered for that particular region of the **behaviour space**.

Definition: Behaviour Space and Behaviour Descriptors

The **behaviour space** is a multi-dimensional space whose axes are *solution characteristics*—measurable properties of a solution that describe *how* it behaves, rather than how good it is. Examples: the total financial cost c , the CO₂ emissions e , the fraction of deliveries made by cargo bike. A **behaviour descriptor** is a specific choice of such characteristics used to define the axes of the archive grid.

A concrete example. Suppose a logistics problem has two solution characteristics: financial cost c and environmental impact (CO₂ emissions) e . Imagine we evolved the following four solutions:

6.2 MAP-Elites: Core Idea II

c (cost)	e (emissions)
500	1023
550	1050
610	990
490	1120

6.2 MAP-Elites: Core Idea III

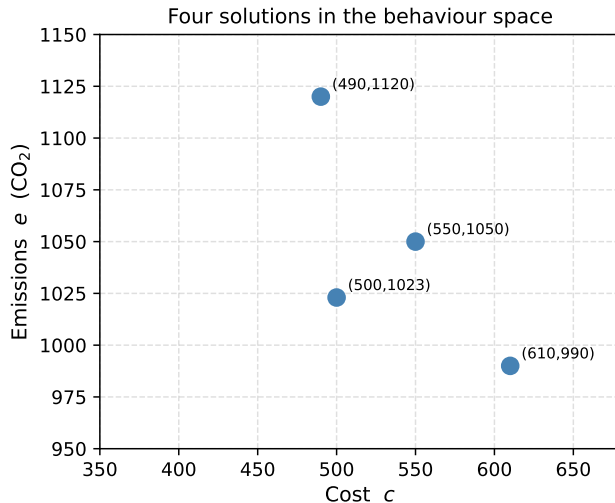


Figure: The four solutions plotted in the two-dimensional behaviour space (c on the horizontal axis, e on the vertical axis). Each point represents one complete delivery plan

6.2 MAP-Elites: The Bin-Scaling Formula I

To place a solution into the archive, each raw characteristic value r must be translated into a **scaled bin index** s . The following three-step formula performs this translation.

Bin-Scaling Formula

Let r be the raw value of one characteristic, let min and max be the known minimum and maximum values of that characteristic, and let b be the number of bins per axis. Then:

$$\delta = (max + 1) - min$$

$$cap = \frac{\delta}{b}$$

$$s = \text{int}\left(\frac{r - min}{cap} + 1\right)$$

6.2 MAP-Elites: The Bin-Scaling Formula II

Notation

- δ (delta) is the *total range* of the characteristic (number of distinct integer values it can take). Adding 1 ensures the maximum value maps to bin b rather than falling outside.
- cap (capacity) is the *width* of each bin—how many raw values fall into each cell.
- b is the number of bins per dimension (e.g., $b = 20$).
- s is the resulting bin index (from 1 to b). The function $\text{int}(\cdot)$ truncates to the nearest integer below (floor).

6.2 MAP-Elites: The Bin-Scaling Formula III

How to Read the Scaling Formula

The formula maps any value in $[min, max]$ uniformly onto $\{1, 2, \dots, b\}$. The larger r is, the larger s is—the relationship is linear. If we double b (use finer bins), cap halves and the archive becomes more detailed but sparser (fewer solutions per cell on average). Choosing b is a design trade-off: large b gives fine resolution but requires more evaluations to fill the archive; small b fills quickly but loses detail.

6.2 MAP-Elites: The Bin-Scaling Formula IV

Worked example: mapping raw value $r = 500$ to bin $s = 10$
($min = 350$, $max = 650$, $b = 20$ bins)

Step 1 — compute range: $\delta = (max + 1) - min = (650 + 1) - 350 = 301$

Step 2 — bin capacity: $cap = \delta / b = 301 / 20 = 15.05$

Step 3 — scaled index: $s = \text{int}\left(\frac{r - min}{cap} + 1\right) = \text{int}\left(\frac{500 - 350}{15.05} + 1\right) = \text{int}(10.97) = 10$

6.2 MAP-Elites: The Bin-Scaling Formula V

Figure: Worked example: the raw cost value $r = 500$ with $min = 350$, $max = 650$, and $b = 20$ bins maps to scaled bin index $s = 10$.

6.2 MAP-Elites: The Bin-Scaling Formula VI

Worked example for all four solutions. Using $c \in [350, 650]$ and $e \in [950, 1140]$, with $b = 20$:

Raw values		Bin index	
c	e	s_c	s_e
500	1023	10	8
550	1050	14	11
610	990	18	5
490	1120	10	18

The four solutions occupy four different cells in the 20×20 archive grid:

6.2 MAP-Elites: The Bin-Scaling Formula VII

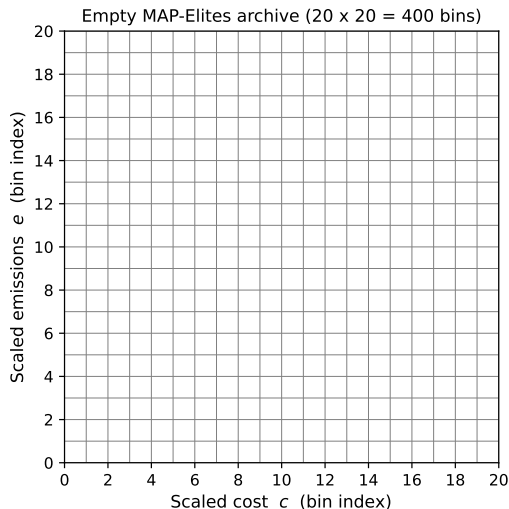


Figure: The empty 400-cell archive, ready to

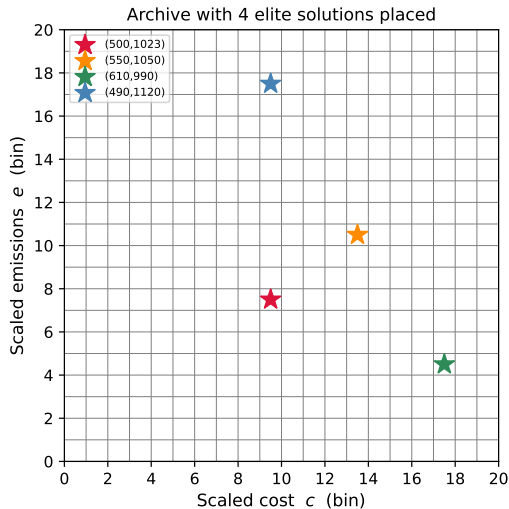


Figure: After placing the four solutions: each star

6.2 MAP-Elites: The Fitness Value and Archive Update I

The behaviour descriptors tell us *which cell* a solution belongs to. But two different solutions can map to the same cell. To decide which solution gets to occupy a cell, MAP-Elites uses a **fitness value** f .

Fitness in MAP-Elites

The fitness value f measures the *quality* of a solution, independently of its behaviour descriptors. In the delivery problem, f could be the total distance travelled: shorter routes are better (lower f). The archive update rule is: “a new solution n replaces the current occupant of a cell only if $f(n) < f(\text{current})$ ”. This ensures every cell always holds the *best* solution found so far for that region of the behaviour space.

Intuition. Think of each cell as a mini-competition. Two solutions with the same (cost-level, emissions-level) combination compete directly: whichever has the lower total distance wins the cell. Solutions in different cells do not compete—they represent genuinely different trade-offs.

6.2 MAP-Elites: The Fitness Value and Archive Update II

Worked Example: Cell Collision

Suppose cell $(10, 8)$ is already occupied by a solution with $c = 500$, $e = 1023$, total distance $f = 340$ km. A new solution arrives with $c = 512$, $e = 1020$, total distance $f = 330$ km. Both map to cell $(10, 8)$ (same bin indices under our scaling). Since $330 < 340$, the new solution *replaces* the old one. The archive now holds a slightly better solution in that cell.

6.2 MAP-Elites: The Fitness Value and Archive Update III

Key Insight: Decoupling quality from diversity

In MAP-Elites, quality (fitness) and diversity (cell location) are *decoupled*. The fitness function evaluates quality *within* a cell. The behaviour descriptors decide *which* cell. This is fundamentally different from a Pareto approach, where the same objectives serve both purposes simultaneously. The decoupling gives the algorithm designer much more flexibility in choosing what counts as “good” vs. what counts as “different”.

6.2 MAP-Elites: The Algorithm (Algorithm 14) I

The MAP-Elites algorithm is conceptually simple: initialise with random solutions, then repeatedly generate new candidates (by recombination or mutation), evaluate them, and attempt to add them to the archive. The algorithm continues until a total-evaluations budget `totEvals` is exhausted.

6.2 MAP-Elites: The Algorithm (Algorithm 14) II

Algorithm 14: MAP-Elites (pseudocode)

- 1 Set *buckets* = 20, *dimensions* = 2, *evals* = 0.
- 2 Initialise the map (archive).
- 3 **Initialisation phase:** while *evals* < *init*:
 - 1 $n \leftarrow \text{newIndividual}()$ (random solution)
 - 2 *evals* += 1
 - 3 add(*map*, *n*)
- 4 **Main loop:** while *evals* < *totEvals*:
 - 1 If random() < *xoverPressure*:
 $c \leftarrow \text{newIndividual}(\text{map.random}(), \text{map.random}())$ (crossover two random parents from archive)
 - 2 Else:
 $c \leftarrow \text{newIndividual}(\text{map.random}())$ (clone one random parent)
 - 3 *c*.mutate() (always apply mutation)
 - 4 add(*map*, *c*)
- 5 Return the archive.

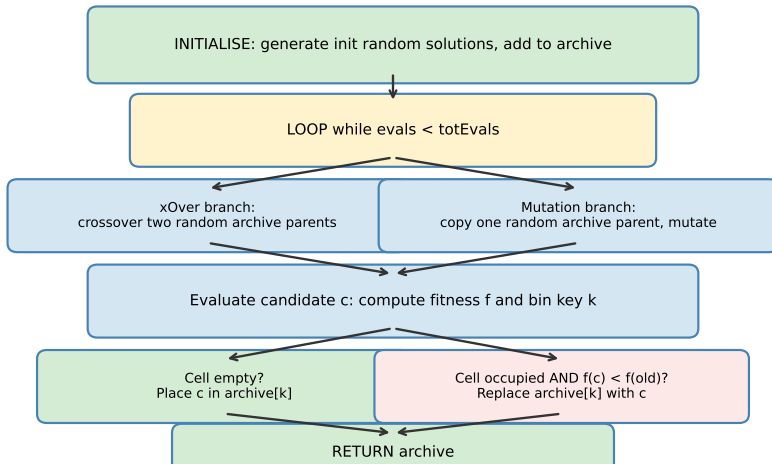
6.2 MAP-Elites: The Algorithm (Algorithm 14) III

The add procedure. Adding a solution to the archive requires just three steps:

- 1 Compute the *key* (bin index tuple) for the solution:
 $key \leftarrow \text{getKey}(n)$ (apply the bin-scaling formula to each characteristic)
- 2 If `map.get(key) == null`: the cell is empty, so place the solution directly: `map.put(key, n)`.
- 3 Else: the cell is occupied. Let $old = \text{map.get}(key)$. If `new.fitness() < old.fitness()`: replace the occupant: `map.put(key, n)`.

6.2 MAP-Elites: The Algorithm (Algorithm 14) IV

MAP-Elites Algorithm (Algorithm 14) — control flow



6.2 MAP-Elites: The Algorithm (Algorithm 14) V

Figure: Control flow of the MAP-Elites algorithm. After initialisation, the main loop alternates between generating candidates (crossover or clone+mutate) and attempting to add them to the archive.

6.2 MAP-Elites: The Algorithm (Algorithm 14) VI

Two key design parameters.

(1) xOverPressure controls the fraction of candidates generated by crossover versus mutation. A value of 0.5 means crossover and mutation are equally likely. Higher values bias the search towards combining existing archive solutions; lower values rely more on random mutation.

(2) totEvals (total evaluations) is the computational budget. Each call to evaluate a candidate costs one evaluation. More evaluations generally means a fuller, higher-quality archive, but also more computation time.

Key Insight: The archive as both population and result

In a standard evolutionary algorithm, the population holds candidate solutions during the search and is discarded at the end. In MAP-Elites, the archive *is* the population *and* the final result. At every point during the run, the archive is a complete, usable set of diverse elite solutions. The algorithm never “loses” diversity by replacing the whole population—once a cell is filled, that solution is preserved until something strictly better comes along.

6.2 Archive Capacity and Sparse Storage I

With b bins per dimension and d dimensions, the archive has a theoretical capacity of:

Archive Capacity Formula

$$c = b^d$$

where c is the total number of cells, b (the bucket count) is the number of bins per dimension (axis), and d is the number of dimensions (solution characteristics used as behaviour descriptors). For $b = 20$ and $d = 2$: $c = 20^2 = 400$. For $b = 10$ and $d = 9$: $c = 10^9 = 1,000,000,000$.

How to Read This: Exponential growth

The capacity grows *exponentially* with the number of dimensions. Doubling the number of dimensions squares the capacity. This is the famous “**curse of dimensionality**”: adding even one more behaviour descriptor multiplies the required archive storage by b . For $b = 10$, going from 8 to 9 dimensions multiplies the archive by 10, from 10^8 to 10^9 cells.

6.2 Archive Capacity and Sparse Storage II

In practice, for the FabFoods/supermarket delivery problem with 9 characteristics and $b = 10$, the theoretical archive size is 10^9 . Creating a Java array of 10^9 slots would exhaust JVM heap memory. The solution is a **sparse map (dictionary)**: instead of pre-allocating all cells, the archive is stored as a hash map that only allocates memory for *occupied* cells. As solutions are found, new entries are added on demand. This way a sparse archive of 500 occupied cells uses only 500 entries, not 10^9 .

The cells are identified by **keys**: arrays of integers, one value per dimension, each value being the scaled bin index of the solution's characteristic. For example, the key $[10, 8]$ identifies the cell at scaled-cost bin 10, scaled-emissions bin 8.

6.3 The Delivery Planning Problem I

The scenario studied in this chapter extends the supermarket delivery problem introduced in Chapter 5. The supermarket now operates a **mixed fleet**: conventional vans *and* cargo bikes. A new delivery plan must be generated for each working day, because the number of deliveries and their locations change with customer demand.

Problem: Multi-Mode Supermarket Delivery

Given a set of customers with delivery time windows and quantity requirements, and a depot, plan routes for vans and cargo bikes such that:

- every customer is served within their time window;
- vehicle capacity constraints are respected;
- a rich archive of diverse, high-quality plans is produced, covering the full range of cost/emissions/bike-usage trade-offs.

Why add cargo bikes? Cargo bikes produce zero CO₂ emissions and have low running costs, making them attractive for sustainability-conscious businesses. However, they have much smaller capacity and

6.3 The Delivery Planning Problem II

lower speed than vans. Depending on the day's delivery profile, the optimal mix of vans and bikes will vary. MAP-Elites builds an archive that shows the manager the full range of van/bike combinations and their associated costs and emissions, so that the manager—not the algorithm—decides how green to be on any given day.

6.3 The Cost Model (Table 6.2) I

A **cost model** is needed to evaluate solutions realistically. The model assigns costs based on the vehicles used, the distance travelled, the time taken, and the number of reload trips. The variables and values used in this chapter are shown below.

6.3 The Cost Model (Table 6.2) II

Delivery Cost Model (Table 6.2)

Parameter	Van	Cargo Bike
Capacity (units)	100	20
Fixed cost per day (£)	164	16
Running cost per km (£)	0.117	0.010
Staff cost per hour (£)	12	12
Loading time between runs (min)	30	15
Speed (km/h)	10	1
Emissions (g CO ₂ /km)	158	0

6.3 The Cost Model (Table 6.2) III

Notation and Meaning

- **Fixed cost per day:** paid regardless of how much the vehicle is used. A van parked all day still costs £164; a bike costs £16.
- **Running cost per km:** fuel/maintenance cost proportional to distance driven. A van costs about 12× more per km than a bike to run.
- **Staff cost per hour:** the same for both modes (£12), because a human operator rides both.
- **Loading time:** the time between completing one run and starting the next (reload + dispatch). Vans take 30 minutes to reload; bikes take 15 minutes.
- **Speed:** vans are 10× faster than bikes (10 km/h vs. 1 km/h), which dramatically affects how many customers a bike can serve per day.
- **Emissions:** vans emit 158 g CO₂/km; bikes emit zero.

6.3 The Cost Model (Table 6.2) IV

How to Read the Cost Model

These numbers reveal a fundamental trade-off: bikes are *much* cheaper to run and produce zero emissions, but they are 10× slower and carry only one-fifth the capacity of a van. A bike that takes 5 hours to complete one delivery run (at 1 km/h, covering perhaps 5 km) might do the same work as a van in 30 minutes. The time difference translates directly into *staff cost*, which is the dominant cost for slow vehicles. The optimal vehicle mix therefore depends on the spatial distribution and quantities of today's deliveries.

Nine solution characteristics (behaviour descriptors) are tracked for each solution (Table 6.1):

- 1 Total daily fixed cost
- 2 Total staff cost
- 3 Total vehicle running cost
- 4 Cost per delivery (per unit crate)
- 5 Emissions produced (g CO₂)
- 6 Percentage of deliveries made by bike

6.3 The Cost Model (Table 6.2) V

- 7 Percentage of distance covered by bike
- 8 Number of bicycles required
- 9 Number of vans required

These nine characteristics form the axes of the MAP-Elites archive. With $b = 10$ bins per axis, the archive has 10^9 possible cells, stored sparsely as described earlier.

6.3 Solution Characteristics (Table 6.1) I

The nine characteristics serve two roles simultaneously: they are **behaviour descriptors** (they define the cell in the archive) and **decision criteria** (the manager uses them to choose a plan). This dual role is one of the key design insights in this chapter.

Why nine characteristics?

A traditional bi-objective formulation would pick two objectives and trade them off. But real logistics managers care about many things at once: a financial director cares about total cost; an operations manager cares about how many vehicles are needed; a sustainability officer cares about emissions and bike usage. By tracking all nine as archive dimensions, MAP-Elites produces a map that serves all stakeholders without requiring anyone to pre-specify their priorities.

Characteristics 1–4 are **cost-related**: they can all be reduced independently (fixed costs from fewer vehicles, staff costs from faster routes, running costs from shorter distances, delivery cost from better load efficiency). Because these may come from different budget lines, a manager may care about one more than another.

6.3 Solution Characteristics (Table 6.1) II

Characteristic 5 (**emissions**) is driven primarily by van usage: each van kilometre produces 158 g CO₂, while bikes produce none.

Characteristics 6–7 (**bike fractions**) capture the modal split: what fraction of the workload is done by bike. These are closely related to emissions but focus on the operational picture.

Characteristics 8–9 (**vehicle counts**) tell the manager how many physical assets are needed, which determines feasibility on days when staff or vehicles are unavailable.

6.3 Representation and Decoding I

The MAP-Elites algorithm uses an **evolutionary algorithm (EA)** to generate candidate solutions. Each candidate is encoded as a **chromosome**: a list of genes, one per customer. Each gene contains three items: (1) the customer ID, (2) a flag indicating whether this customer starts a new route (0 = add to current route, 1 = start a new route), and (3) the delivery mode (V = van, B = cargo bike).

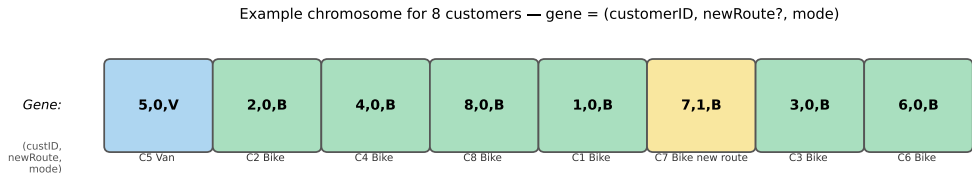


Figure: An example chromosome for 8 customers. The first gene places customer 5 on route 1 using a van. Gene “7,1,B” starts a new route (flag=1) for customer 7 using a cargo bike.

6.3 Representation and Decoding II

Decoding: from chromosome to routes

The **decoder** reads genes left-to-right and builds routes:

- 1 If the new-route flag is 0 and the current customer fits in the current route (time window and capacity), append to current route.
- 2 If the flag is 1, or the customer does not fit, start a new route.
- 3 The *mode* of a route is determined by the first customer's mode gene: if that gene says "V", the route uses a van; if "B", the route uses a cargo bike.

6.3 Representation and Decoding III

Worked decoding example. Given the chromosome 5,0,V | 2,0,B | 4,0,B | 8,0,B | 1,0,B | 7,1,B | 3,0,B | 6,0,B and the following customer data (time windows, travel times in format *van/bike*):

- ① **Gene 1 (5,0,V):** Start route 1 using a van. Customer 5 has time window 12:00–13:00. Initial arrival time 08:38 (based on depot travel time), but window opens at 12:00, so the van waits and arrives at 12:00. Routes so far: Route 1V: [5@12:00]
- ② **Gene 2 (2,0,B):** Customer 2 has time window 09:00–10:00. The earliest arrival from customer 5 at 12:05 is after 10:00 (the window closes at 10:00), so customer 2 cannot join route 1. Because customer 2's mode gene is "B", a new bike route is started: Route 2B: [2@09:00].
- ③ **Gene 3 (4,0,B):** Customer 4's window is 13:00–14:00. It fits on route 2B after customer 2. Route 2B: [2@09:00, 4@13:00].
- ④ **Gene 4 (8,0,B):** Route 2B is now at capacity (20 units for a bike). The bike returns to the depot (arriving 13:40), reloads (15 min), departs 13:50. Customer 8 is added to a new sub-run of route 2B: [2@09:00, 4@13:00, D, 8@14:01].

6.3 Representation and Decoding IV

- 5 **Gene 5 (1,0,B)**: Customer 1 has window 09:00–10:00. Earliest arrival is after 10:00, so a new van route is started. This continues until the full chromosome is decoded.

The resulting routes are then evaluated using the cost model to produce the fitness value f (total distance) and all nine solution characteristics.

6.3 Setting Archive Boundaries: FindBoundaries I

Before MAP-Elites can scale characteristics into bin indices, it needs to know the *range* of each characteristic: what are the minimum and maximum plausible values? For some characteristics (like the fraction of deliveries by bike), the range is obvious: $[0, 1]$. For others (like total staff cost), the range depends on the specific problem instance and is not known in advance.

Algorithm 15: FindBoundaries

Method 1 (optimise each objective separately):

- 1 For each characteristic o : run a separate EA to *optimise* (minimise/maximise) o .
- 2 Use the resulting population to update the known range of o .

Method 2 (scan a set of solutions):

- 1 For each solution in the set:
- 2 For each characteristic o : if solution's value for o is less than known lower bound, update lower bound. If greater than known upper bound, update upper bound.

6.3 Setting Archive Boundaries: FindBoundaries II

Notation

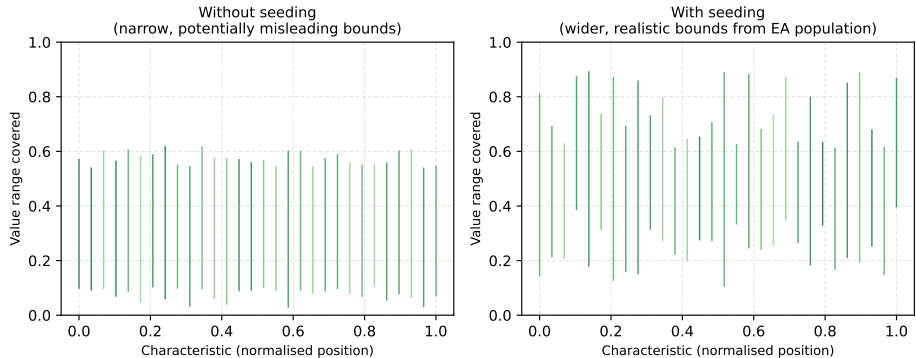
- *objective.lower* and *objective.upper*: the current known minimum and maximum for that characteristic.
- These bounds determine *min* and *max* in the bin-scaling formula, so getting them right is critical for archive quality.

6.3 Setting Archive Boundaries: FindBoundaries III

The seeding technique. Table 6.3 shows that for characteristics 1–4 (cost-related) and 8–9 (vehicle counts), the upper bounds are marked “?” because they cannot be determined analytically from an unsolved instance. The **seeding** technique addresses this: run 10 independent short EA runs, collect the final populations, and feed all solutions into FindBoundaries Method 2. The EA solutions tend to cover a wide range of the characteristics, giving realistic bounds.

6.3 Setting Archive Boundaries: FindBoundaries IV

Effect of seeding on archive boundary estimation (Fig. 6.2)



6.3 Setting Archive Boundaries: FindBoundaries V

Figure: Schematic illustration of the seeding effect. Without seeding (left), the archive has narrow, possibly misleading bounds. With seeding (right), the bounds are wider and more realistic, allowing the archive to represent the true diversity of the solution space.

6.3 Setting Archive Boundaries: FindBoundaries VI

Why seeding matters

If the bounds are set too tight, good solutions outside the range will be *clipped* to the boundary bins. Multiple solutions collapse into the same boundary cells, reducing diversity. If bounds are set too wide, the bins are very coarse and many qualitatively different solutions map to the same cell—again reducing diversity. Seeding gives a data-driven estimate of realistic bounds from actual solutions, striking the right balance.

The actual parameter ranges found by seeding for the supermarket delivery problem are shown in the book (Table 6.4). For example, for **CostDel** (cost per delivery):

- Seeded: lower = 1.18, upper = 11.9 $\Rightarrow \delta \approx 10.72$, bin size ≈ 1.072
- Unseeded: lower = 0, upper = 7.65 $\Rightarrow \delta = 7.65$, bin size ≈ 0.765

The seeded range is about 40% wider, meaning the archive covers a broader region of the cost-per-delivery axis.

6.3 Java Implementation: The MAP-Elites Solver (Listing 6.1)

The book provides a Java implementation of MAP-Elites. The core `solve()` method is shown below. Key classes: `Archive` (sparse hash map of elite solutions), `EliteSolution` (extends the EA's individual class with fitness/key methods), `EliteSolutionFactory` (creates new solutions by recombination or cloning).

```
public void solve() {
    archive = new Archive(KeyGenerator.getInstance().getDimensions(),
                          KeyGenerator.getInstance().getBuckets());

    initialise();
    EliteSolution ch;
    for (int c=0; c < MAX_EVALS; c++) {
        if ((c%10000)==0) {
            System.out.print(" " + c + " : " + archive.size());
        }
        if (rnd.getRnd().nextBoolean()) {
            // Select two random parents from the archive
            EliteSolution p1 = (EliteSolution)archive.getRandom();
            EliteSolution p2 = (EliteSolution)archive.getRandom();
            // Create a new solution from the parents (crossover)
            ch = (EliteSolution) solutionFactory.getChild(theProblem, p1, p2);
            if (rnd.getRnd().nextBoolean()) {
                ch.mutate();           // Mutate the child solution
            }
        } else {
            // Create a new solution by copying an existing archive member
            ch = solutionFactory.copy((EliteSolution)archive.getRandom());
            ch.evaluate();
        }
    }
}
```

6.3 Python Equivalent: MAP-Elites Core Logic I

Below is a Python implementation of the MAP-Elites core logic, demonstrating the same ideas as the Java listing with a simpler delivery toy problem. This Python version is self-contained and can be run directly.

Python: MAP-Elites Core (Part 1 — Scaling and Archive)

```
import math, random

def scale_to_bin(r, rmin, rmax, b=10):
    """Map raw value r in [rmin, rmax] to bin index 1..b."""
    delta = (rmax + 1) - rmin
    cap    = delta / b
    s      = int((r - rmin) / cap + 1)
    return max(1, min(b, s))    # clamp to valid range

def get_key(solution, descriptors, bounds, b=10):
    """
    Compute the archive key (tuple of bin indices) for a solution.
    solution      : dict of characteristic name -> value
    descriptors    : list of characteristic names to use as archive axes
    bounds        : dict of name -> (rmin, rmax)
    """
    key = tuple(
        scale_to_bin(solution[name], bounds[name][0], bounds[name][1], b)
        for name in descriptors
    )
    return key

def archive_add(archive, solution, key):
    """
    Add solution to the archive at position key.
    Returns True if the archive was updated (cell improved or filled).
    Lower fitness is better.
    """
```

Python: MAP-Elites Core (Part 2 — Main Loop)

```
def map_elites(problem, tot_evals=50_000, xover_pressure=0.5, b=10):
    """
    Run MAP-Elites on 'problem'.
    problem must provide: random_solution(), mutate(sol),
    crossover(sol1, sol2), evaluate(sol), descriptors, bounds.
    Returns the archive (dict: key -> solution).
    """
    archive = {}

    # --- Initialisation phase: 100 random solutions ---
    for _ in range(100):
        sol = problem.random_solution()
        problem.evaluate(sol)
        key = get_key(sol, problem.descriptors, problem.bounds, b)
        archive_add(archive, sol, key)

    # --- Main search loop ---
    for ev in range(100, tot_evals):
        if random.random() < xover_pressure:
            # Crossover: combine two random archive members
            p1 = random.choice(list(archive.values()))
            p2 = random.choice(list(archive.values()))
            child = problem.crossover(p1, p2)
        else:
            # Clone + mutate a random archive member
            child = dict(random.choice(list(archive.values())))
```

6.4 Results: Experimental Setup I

The MAP-Elites algorithm was evaluated on the **Augerat benchmark instances**: three sets (A, B, P) of vehicle routing problems with time windows, ranging from small (12 customers) to large (101 customers). Each problem was solved 10 times independently to obtain reliable statistics.

Experimental setup

- **Problem instances**: Augerat sets A (27 problems), B (24 problems), P (24 problems) with time windows.
- **Algorithm**: MAP-Elites with $b = 10$ bins, 9 characteristics, sparse hash-map archive.
- **Comparison**: Seeded vs. unseeded boundary estimation.
- **Metrics**: archive size (number of occupied cells), best fitness found per run, number of feasible solutions.
- **Scenarios**: A1–A4 (aspirations applied to filter the archive), C1–C3 (constraints applied), and combinations.

The **costing model** assigns a total cost per solution:

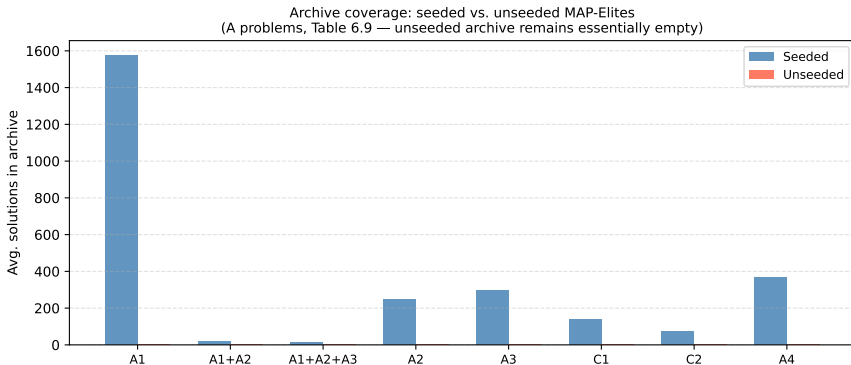
6.4 Results: Experimental Setup II

- Fixed cost per vehicle per day (van: £164, bike: £16).
- Running cost per km (van: £0.117, bike: £0.01).
- Staff cost at £12/hour for all travel and loading time.
- Loading time at 30 min (van) or 15 min (bike) between runs.

The fitness value f used within MAP-Elites is the total distance (in km), which is a good proxy for overall cost and is computationally cheap to compute. The nine solution characteristics are derived from f and the routing details.

6.4 Results: Archive Coverage I

Archive coverage measures how many cells in the archive contain at least one elite solution. High coverage means MAP-Elites has found solutions representing a wide variety of trade-offs. Low coverage means the algorithm has not fully explored the behaviour space.



6.4 Results: Archive Coverage II

Figure: Average archive coverage (number of solutions in the archive) for seeded vs. unseeded MAP-Elites across selected scenarios (A problems, Table 6.9). Seeding consistently produces archives with many more solutions; unseeded archives remain nearly empty.

6.4 Results: Archive Coverage III

Key finding: Seeding is essential

Table 6.9 in the book shows that when no constraints or aspirations are applied (scenario A1, meaning “only aspiration 1 is active”), the seeded MAP-Elites finds an average of 1,576.7 solutions per problem instance for A problems, while the unseeded version finds essentially zero. The seeded version also dominates in terms of “seeded wins” (the seeded archive contains a strictly better solution than the unseeded archive in almost every problem).

Why does unseeded fail so completely? Without seeding, the archive uses tight, possibly incorrect bounds. Solutions that the EA generates may map to boundary cells (because their characteristic values fall outside the unseeded range), causing many solutions to compete for the same few cells. The effective archive becomes very small, the algorithm degenerates into a standard single-population EA, and diversity collapses.

6.4 Results: Archive Coverage IV

Key Insight: The boundary estimation determines archive quality

The quality of a MAP-Elites run depends critically on how well the archive boundaries are set. Seeding uses actual evolutionary algorithm solutions to set realistic bounds, and this alone transforms the algorithm from ineffective to highly effective. This is a practical lesson: *always* seed your MAP-Elites boundaries from domain knowledge or preliminary runs.

6.4 Results: Constraints and Aspirations I

One of the most powerful features of the MAP-Elites archive is that a manager can **interactively filter** it in real time by applying **constraints** and **aspirations**, without re-running the algorithm.

Constraints (C): hard limits

A constraint removes from consideration any solution that violates a hard limit.

- **C1 — Vehicle shortage:** only 2 vans can be operated on this day (e.g., due to maintenance). Remove all archive solutions that use more than 2 vans.
- **C2 — Cargo bike shortage:** only 2 cargo bikes are available. Remove all archive solutions requiring more than 2 bikes.
- **C3 — Emissions cap:** a regulatory or corporate emissions ceiling applies. Remove all solutions exceeding the cap.

Constraints are applied as filters over the archive and do not change the archive itself—the removed solutions remain available and can be restored if the constraint is relaxed.

6.4 Results: Constraints and Aspirations II

Aspirations (A): soft preferences

An aspiration is a preference to move in a certain direction in the solution space.

- **A1**: Reduce staff cost and running cost (minimise operational expenditure).
- **A2**: Maximise the fraction of deliveries made by cargo bike (sustainability preference).
- **A3**: Reduce fixed vehicle costs (use fewer or cheaper vehicles).
- **A4**: Minimise cost per delivery unit.

Aspirations are applied by restricting the axis ranges in the archive, effectively zooming in to the region of the archive that meets the aspiration.

6.4 Results: Constraints and Aspirations III

Interactive filtering workflow.

- ➊ Run MAP-Elites once to produce the full archive (may take minutes to hours depending on problem size).
- ➋ Present the archive to the manager as a parallel coordinates plot, with each axis corresponding to one characteristic.
- ➌ The manager applies a constraint (e.g., “I only have 2 vans today”): all solutions with more than 2 vans are greyed out.
- ➍ The manager adds an aspiration (e.g., “I want to minimise running cost”): the archive is filtered to show only solutions below a running-cost threshold.
- ➎ The manager browses the remaining solutions and selects one.

This interactive process happens in **seconds**, because it is just filtering a pre-built data structure.

6.4 Results: Constraints and Aspirations IV

Fig. 6.3 Archive as parallel coordinates (B-n67-k10, 1h time windows)

122

6 Illuminating Problems

The next delivery for consideration is 8, this can be allocated to route 2, as the bicycle will arrive at 14:01.

The next customer for consideration is customer 1, as they have a 09:00 to 10:00 window they cannot be served on route 2, so another route is started using a van. This continues until the genome has been fully decoded.

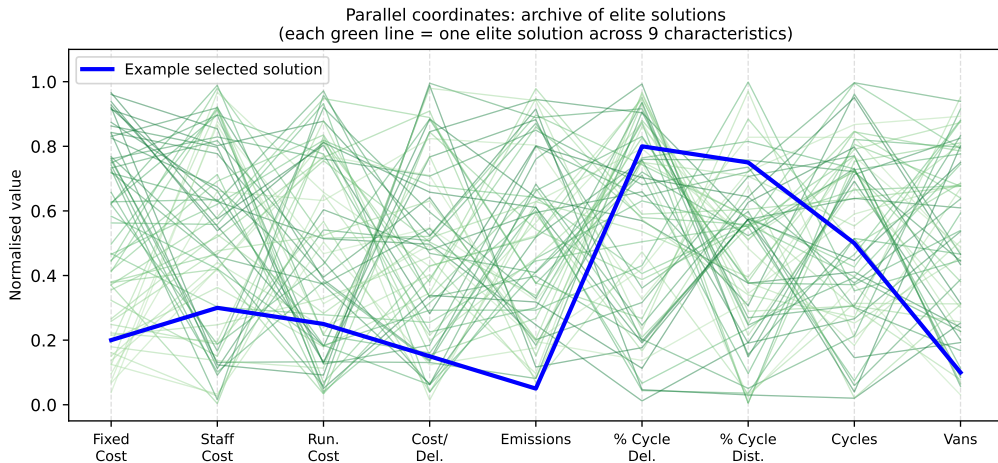
Routes			
1V		2B	
5	13:00	2	09:00
		4	13:00
		D	
		8	14:01

Implementation

6.4 Results: Parallel Coordinates Visualisation I

Parallel coordinates plots provide a way to visualise and interact with the high-dimensional archive. Each vertical axis represents one solution characteristic. Each horizontal polyline (connecting all axes) represents one elite solution in the archive. A manager can “brush” an axis (restrict its range) to filter solutions interactively.

6.4 Results: Parallel Coordinates Visualisation II



6.4 Results: Parallel Coordinates Visualisation III

Figure: Illustrative parallel coordinates plot of an elite archive with 9 characteristics. Each green line represents one elite solution. Darker green lines correspond to solutions with lower fixed cost. The highlighted blue line is an example selected solution with low emissions (axis 5 near 0) and high bike usage (axes 6–7 near top).

How to read a parallel coordinates plot

- Each **vertical axis** represents one characteristic. Values increase from bottom to top.
- Each **line** (polyline) is one solution. A line that stays low on the “Emissions” axis and high on the “%Cycle Deliveries” axis represents a predominantly bike-based, low-emission plan.
- **Crossings** between lines near an axis indicate that two solutions have very different values for that characteristic but converge elsewhere—they represent qualitatively different plans with similar overall cost.
- **Brushing** (restricting the range of an axis) is the interactive filtering operation: it corresponds to applying a constraint or aspiration.

6.4 Results: Summary Tables (A, B, P Problems) I

The results across all three benchmark sets (A, B, P problems from Tables 6.5–6.7) show consistent patterns:

6.4 Results: Summary Tables (A, B, P Problems) II

Summary findings from Tables 6.5–6.7

- **Fitness quality:** the seeded and unseeded versions produce similar *best fitness* values when they do find solutions. The primary advantage of seeding is in archive *coverage* (how many cells are filled), not in the quality of individual solutions.
- **Problem scaling:** larger problems (more customers) generally produce larger archives, because there is more solution diversity possible. Problem P-n101-k4-1 (101 customers, 4 vehicles) produces some of the largest archives.
- **Bike usage in unseeded results:** interestingly, in all presented results, the percentage of cycle deliveries and cycle distance is 0/0 in both seeded and unseeded runs. This suggests that under the cost model, cargo bikes are rarely chosen by the EA as the default mode, and the MAP-Elites archive in the main runs focuses primarily on van-based plans. The seeding and archive structure allow the *presence* of bike solutions, but the fitness function (total distance) may not sufficiently incentivise bike usage.

6.4 Results: Summary Tables (A, B, P Problems) III

Example from Table 6.5 (A problems, selected rows)

For problem A-n32-k5-1 (32 customers, 5 vehicles):

- Archive size: 5047 total solutions, average fitness 5285.5 (seeded), 5179.1 (unseeded).
- Fixed cost: £288 (seeded run) to £304 (unseeded).
- Staff cost: £181–£142.6, running cost: £46–£59.
- Cost per delivery: £2.29–£2.28 per unit.

This shows the range of costs across the 10 independent runs, illustrating the natural variability in the evolutionary process.

6.5 Conclusions I

This chapter has demonstrated how **MAP-Elites**, an illumination algorithm from the Quality-Diversity family, can be applied to the multi-mode supermarket delivery problem to produce a rich, browsable archive of high-quality delivery plans.

What MAP-Elites achieves

- It produces a **structured collection** of diverse plans rather than a single solution, giving the operations manager genuine choice.
- The archive can be **filtered interactively** (in seconds) by applying constraints (hard operational limits) and aspirations (soft preferences), without re-running the algorithm.
- When properly seeded, the archive provides meaningful choices even under multiple simultaneous constraints.

6.5 Conclusions II

Design lessons

- 1 **Seed the boundaries:** always use a preliminary evolutionary run (seeding) to estimate archive bounds. Unseeded archives remain nearly empty, defeating the purpose of MAP-Elites.
- 2 **Choose behaviour descriptors carefully:** the nine characteristics used here were selected to cover the key concerns of all stakeholders (financial, operational, environmental). Choosing irrelevant characteristics wastes archive capacity and increases computational cost.
- 3 **Sparse storage is essential** for high-dimensional archives (9 dimensions with $b = 10$ gives 10^9 possible cells; a dense array would require terabytes). A hash map that only stores occupied cells scales gracefully.
- 4 **The fitness function and descriptors are decoupled:** the fitness (total distance) drives within-cell competition; the descriptors (9 characteristics) define which cell. This decoupling is a deliberate and powerful design choice.

6.5 Conclusions III

Key Insight: Illumination vs. optimisation

Classical optimisation asks: “What is the best solution?” MAP-Elites asks: “What is the best solution *for each type of behaviour*?” By answering the second question, MAP-Elites gives the decision-maker far more information and flexibility. The algorithm *illuminates* the problem by showing what is achievable across the full behaviour space, not just at one point.

Limitations and future directions.

- The current results show that cargo bikes are rarely selected under the default fitness function (total distance). Future work could use a multi-objective fitness or an explicit bonus for bike usage.
- The archive building phase (MAP-Elites search) is still computationally expensive for very large instances. Parallel evaluation of candidates could reduce wall-clock time.
- The 9-dimensional archive with $b = 10$ bins gives 10^9 theoretical cells; only a tiny fraction are ever filled. Adaptive binning (varying b by dimension based on solution density) could improve coverage efficiency.

6.5 Conclusions IV

Chapter 6 in one sentence

MAP-Elites illuminates the delivery planning problem by building a sparse archive of elite solutions covering every region of the nine-dimensional behaviour space, enabling real-time interactive selection by the operations manager.

Algorithm Summary: MAP-Elites (Algorithm 14) I

Algorithm Summary: MAP-Elites (Algorithm 14) II

Complete pseudocode

Procedure MAP-Elites(*init*, *totEvals*, *xOverPressure*)

- 1 *buckets* $\leftarrow$ 20; *dimensions* $\leftarrow$ 2; *evals* $\leftarrow$ 0
- 2 *map*.initialise(*buckets*, *dimensions*)
- while** *evals* < *init* **do**
- 3 *n* $\leftarrow$ newIndividual()
- 4 *evals* += 1
- 5 add(*map*, *n*)
- while** *evals* < *totEvals* **do**
- 6 **if** random() < *xOverPressure* **then**
- 7 *c* $\leftarrow$ newIndividual(*map*.random(), *map*.random())
- 8 **else**
- 9 *c* $\leftarrow$ newIndividual(*map*.random())
- 10 *c*.mutate()
- 11 add(*map*, *c*)

Algorithm Summary: FindBoundaries (Algorithm 15) I

Algorithm Summary: FindBoundaries (Algorithm 15) II

Complete pseudocode

Procedure findBoundaries(*problem*, *objectives*)

① **for each** Objective *o* in *objectives* **do**

② *sols* $\leftarrow$ optimise(*o*, *prob*)

③ updateRanges(*sols*)

EndProcedure

Procedure findBoundaries(*solutions*)

④ **for each** Solution *sol* in *solutions* **do**

⑤ **for each** Objective *obj* in *objectives* **do**

⑥ **if** *sol*[*obj*].*lower* < *obj*.*lower* **then**

⑦ *obj*.*lower* $\leftarrow$ *sol*[*obj*].*lower*

⑧ **if** *sol*[*obj*].*upper* > *obj*.*upper* **then**

⑨ *obj*.*upper* $\leftarrow$ *sol*[*obj*].*upper*

EndProcedure

Worked Example: Full MAP-Elites Run (2D Archive) I

This worked example traces through a miniature MAP-Elites run with 2 characteristics (c = cost, e = emissions), $b = 5$ bins each (25 cells), and only 10 candidate solutions for clarity.

Setup

$c \in [400, 700]$: $\delta = 301$, $cap = 60.2$, bins 1–5

$e \in [900, 1200]$: $\delta = 301$, $cap = 60.2$, bins 1–5

Worked Example: Full MAP-Elites Run (2D Archive) II

Candidate solutions and their bin keys

Sol	c	e	f (dist)	s_c	s_e	Key
1	450	950	320	1	1	(1,1)
2	520	980	290	2	2	(2,2)
3	610	1050	340	4	3	(4,3)
4	460	960	310	1	1	(1,1) — collision
5	680	1150	280	5	5	(5,5)
6	460	960	330	1	1	(1,1) — collision
7	530	1100	295	2	4	(2,4)
8	590	1000	305	4	2	(4,2)

Step-by-step archive updates:

- Sol 1 arrives at (1,1): cell empty $\Rightarrow$ place. $f[(1,1)] = 320$.

Worked Example: Full MAP-Elites Run (2D Archive) III

- Sol 2 arrives at (2,2): empty $\Rightarrow$ place. $f[(2,2)] = 290$.
- Sol 3 arrives at (4,3): empty $\Rightarrow$ place. $f[(4,3)] = 340$.
- Sol 4 arrives at (1,1): occupied. $310 < 320 \Rightarrow$ **replace**. $f[(1,1)] = 310$.
- Sol 5 arrives at (5,5): empty $\Rightarrow$ place. $f[(5,5)] = 280$.
- Sol 6 arrives at (1,1): occupied. $330 > 310 \Rightarrow$ **rejected**.
- Sol 7 arrives at (2,4): empty $\Rightarrow$ place. $f[(2,4)] = 295$.
- Sol 8 arrives at (4,2): empty $\Rightarrow$ place. $f[(4,2)] = 305$.

Final archive: 6 cells occupied out of 25. Cell (1,1) holds solution 4 (lower cost, lower emissions, shorter routes than solution 1). Solution 6 was rejected despite having the same cell as solutions 1 and 4, because its distance (330 km) was worse than the current occupant (310 km).

References

- Mouret, J.-B. & Clune, J. (2015). *Illuminating Search Spaces by Mapping Elites*. arXiv:1504.04909. [The original MAP-Elites paper; the foundation of this chapter.]
- Augerat, P. (1995). *Approche polyédrale du problème de tournées de véhicules* (Ph.D. Thesis, Grenoble Institute of Technology). [The benchmark vehicle routing instances used in the experiments.]
- Augerat, P. (2014a/b/c). VRP-REP: Augerat 1995 Sets A, B, P. <http://www.vrp-rep.org/datasets/item/2014-0000.html> [Online datasets used for experiments.]
- Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press. [Foundation of evolutionary algorithms, underlying the EA operators used in MAP-Elites.]
- Inselberg, A. (2009). *Parallel Coordinates: Visual Multidimensional Geometry and Its Applications*. Springer. [The visualisation technique used to present the archive interactively.]

Chapter 6 Summary

Core concepts reviewed

Concept	Key point
Quality-Diversity (QD)	Optimise quality AND coverage simultaneously
MAP-Elites	Archive of best solution per behaviour region
Behaviour descriptors	Solution characteristics defining archive axes
Bin-scaling formula	$s = \text{int}\left(\frac{r - \min}{\text{cap}} + 1\right), \text{cap} = \frac{\delta}{b}$
Archive update rule	Replace cell occupant only if $f_{\text{new}} < f_{\text{old}}$
Archive capacity	$c = b^d$ (exponential in dimensions, use sparse map)
Seeding	Preliminary EA runs set realistic archive bounds
Constraints/Aspirations	Interactive post-hoc filtering in seconds

The single most important idea in Chapter 6

MAP-Elites separates the *search phase* (run once, potentially expensive) from the *decision phase* (run many times, cheap). The archive is the bridge: built once, queried many times, filtered interactively to support real-world decision-making under changing constraints.